Il primo byte identifica l’istruzione

Un’istruzione può occupare 1, 2 o 3 byte. Il primo è sempre l’opcode.

Istruzione	Byte 1: opcode	Byte 2	Byte 3
ADD	`01100111`		
1 byte: solo opcode			
LDI R0, 10	`00100000`	`00001010`	
2 byte: opcode + valore			
JZ 0x1234	`10110000`	`00110100`	`00010010`
3 byte: opcode + indirizzo a 16 bit			

In T1 la CPU legge il primo byte e lo salva nel registro IR.

La CU usa IR[7:3], flag e microstep per generare i segnali di controllo.

L’opcode determina anche quali byte leggere dopo e come utilizzarli.

Da leggere: Prima di vedere le modifiche, ricordiamo come la CPU legge un’istruzione.
Il primo byte identifica l’operazione; gli eventuali byte successivi forniscono gli operandi.

I byte successivi contengono gli operandi

Il loro significato dipende dall’opcode letto nel primo byte.

1 byte: nessun operando aggiuntivo

ADD e CMP usano RA e RB; MOV seleziona il registro nei bit dell’opcode.

Terminata la sequenza, la CPU passa all’istruzione successiva.

2 byte: un valore immediato a 8 bit

LDI R0, 10: il secondo byte è 00001010, cioè il valore da caricare in R0.

Il valore viene letto nei microstep successivi: non è un nuovo opcode.

3 byte: un indirizzo a 16 bit

JZ 0x1234: il secondo byte è 00110100, il terzo è 00010010.

Prima il byte basso (0x34), poi quello alto (0x12): l’indirizzo è 0x1234.

LDA/STA usano l’indirizzo per la memoria; i salti come destinazione del PC.

LDX usa i due byte per caricare il registro indice IDX.

Da leggere: Il secondo e il terzo byte non cambiano il tipo di istruzione: sono dati o indirizzi.
Per il problema della decodifica ci concentriamo quindi sul primo byte, cioè sull’opcode.

Il limite della vecchia codifica

Le EEPROM ricevono IR[7:3]: vedono soltanto i cinque bit alti.

Istruzione	Vecchio opcode	Ingresso CU	Problema
JMP	`10100000`	`10100`	
JZ	`10100001`	`10100`	Stesso gruppo
SUB	`01101000`	`01101`	
CMP	`01101001`	`01101`	Stesso gruppo

I bit che distinguono queste istruzioni sono fuori dalla CU.

Quindi quei cinque ingressi non possono selezionare sequenze diverse.

Da leggere: La CU riceve solo cinque bit dell'istruzione, ma nella vecchia ISA alcune operazioni differivano nei bit esclusi: per le EEPROM erano indistinguibili.

I bit bassi pilotano registri e ALU

Condividere un gruppo CU funziona quando la sequenza di controllo è la stessa.

Categoria | Altri bit | Selezione diretta di registro o ALU

Registri: IR[2:0] seleziona direttamente R0...R7

LDI R0, imm8	**00100000**	**CU: 00100**	Registro: R0
LDI R1, imm8	**00100001**	**CU: 00100**	Registro: R1

La CU esegue lo stesso caricamento; i bit bassi scelgono il registro.

ALU: IR[3:0] seleziona direttamente l'operazione

AND	**01100000**	**CU: 01100**	ALU: 0000
OR	**01100001**	**CU: 01100**	ALU: 0001

La CU salva risultato e flag; il decoder ALU sceglie AND oppure OR.

IR[3] arriva anche alla CU; solo IR[2:0] resta fuori dalle EEPROM.

Per salti e SUB/CMP servono invece decisioni o sequenze CU diverse.

Da leggere: Gli ultimi bit non sono persi: selezionano il registro o l'operazione ALU.
Finché i segnali di controllo sono gli stessi, la CU può usare una sequenza comune.

Nuovi opcode, gruppi CU distinti

Ogni salto può selezionare la propria condizione sui flag.

Istruzione	Opcode attuale	IR[7:3]
JMP	`10100000`	`10100`
JZ	`10110000`	`10110`
JNZ	`10111000`	`10111`

SUB: 01101000 diventa 01111000

Il gruppo CU cambia; IR[3:0] resta 1000 per selezionare la sottrazione.

CMP resta 01101001: salva i flag senza modificare RA.

Da leggere: Ho assegnato a ogni salto un gruppo diverso nei cinque bit visibili alla CU.
Resta un caso particolare: HLT e NOP. Vediamo perché serve una piccola logica esterna.

HLT e NOP: la logica esterna

Le due istruzioni condividono IR[7:3] = 00000: la CU non le distingue.

HLT `00000000` **NOP** `00000001`

La differenza è in IR[0], che non entra nelle EEPROM di controllo.

La CU genera la stessa sequenza: nessuna operazione sui dati.

Ma HLT deve fermare i microstep, mentre NOP deve proseguire al fetch.

In RUN/T2 la CU porta SYS_STEP_n a 0 per entrambe (ROM2, D6).

Una OR esterna usa anche IR[2:0] e genera l'abilitazione del contatore:

`µSTEP_EN = SYS_STEP_n OR IR2 OR IR1 OR IR0`

`HLT: 0 OR 000 = 0`	Il contatore microstep resta fermo.
`NOP: 0 OR 001 = 1`	Il contatore prosegue verso il fetch.

Il segnale agisce sull'abilitazione del 74161; il clock continua a girare.

Da leggere: La CU vede HLT e NOP come la stessa istruzione e produce gli stessi segnali.
La piccola OR cablata legge anche i bit bassi e decide se fermare o far avanzare i microstep.

Confronto ISA: opcode vecchi e attuali

Codifica binaria a 8 bit e ingresso IR[7:3] della Control Unit

Categoria: IR[7:5] · Altri bit: IR[4:0] · Opcode modificato

Istruzione	Vecchio opcode (8 bit)	Vecchio IR[7:3]	Attuale opcode (8 bit)	Attuale IR[7:3]
HLT	00000001	00000	00000000	00000
NOP	00000000	00000	00000001	00000
SUB	01101000	01101	01111000	01111
CMP	01101001	01101	01101001	01101
JMP	10100000	10100	10100000	10100
JZ	10100001	10100	10110000	10110
JNZ	10100010	10100	10111000	10111
JC	10100011	10100	11100000	11100
JNC	10100100	10100	11101000	11101
JN	10100101	10100	11110000	11110
JNN	10100110	10100	11111000	11111
JO	10100111	10100	00110000	00110
JNO	10101000	10101	10101000	10101

Come leggere i colori

Opcode completo: 3 bit gialli + 5 azzurri. Ingresso CU: 3 bit gialli + 2 azzurri (IR[7:3]).

La categoria usa 3 bit; la CU ne riceve 5. I bit IR[2:0] non entrano nelle EEPROM di controllo.

Da leggere: Qui vediamo tutte le modifiche: cambiano gli opcode, ma il significato delle istruzioni resta uguale.

I colori separano la categoria dagli altri bit; ora passiamo al codice che genera le parole nelle EEPROM.

Il codice che genera le EEPROM

Esempio JZ: il gruppo sceglie il salto, il flag Z decide PC_LOAD.

1. Decodifica del gruppo IR

```
// microop_for_ir_group()
case 0x14: return UOP_JMP;
case 0x16: return UOP_JZ;
case 0x17: return UOP_JNZ;
```

0x16 = 10110 in binario

Il codice conta da zero:
step == 3 corrisponde a T4.

2. Parola di controllo a T4

```
// jump_is_taken()
case UOP_JZ:
    return flags.zero;

// microcode_word(), step == 3
return ADDRESS_NONE |
  (jump_is_taken(uop, flags)
    ? CTRL_PC_LOAD : 0);
```

Nelle immagini ROM: JZ + T4 + Z=1 attiva PC_LOAD; Z=0 lo lascia inattivo.

Il C gira sul computer. La CPU legge le parole già scritte nelle EEPROM.

`make -C tools/cu-bytecode clean all test generate`

Riprogramma le tre ROM di controllo e riassembla il programma con la nuova ISA.

Da leggere: Nel generatore, il gruppo identifica JZ e il flag Z decide se attivare PC_LOAD.

Questa scelta viene scritta nelle ROM: ora vi mostro dove avviene nel codice.

Codice: tools/cu-bytecode/src/microcode.c, righe 45-68, 79-105, 334-343 e 648-692